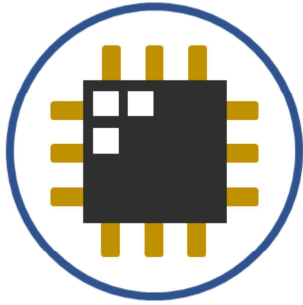
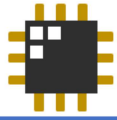




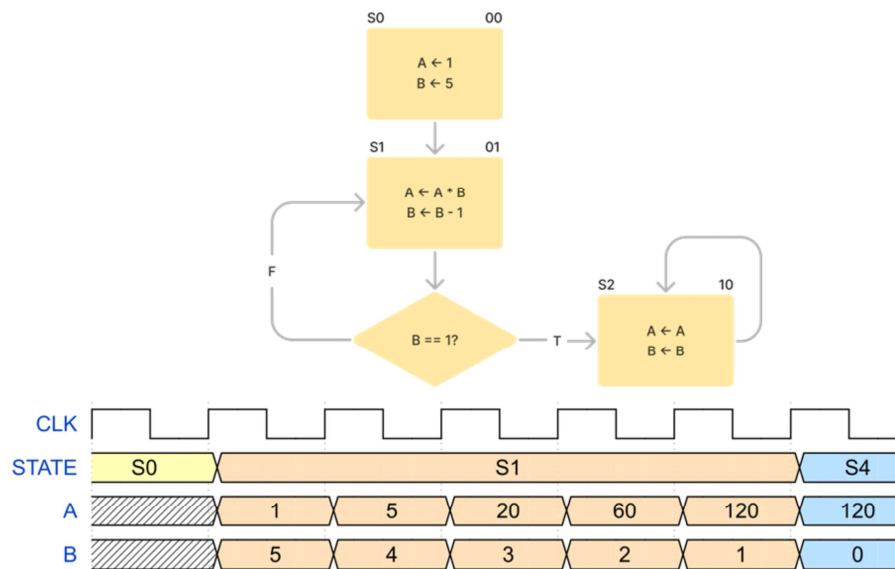
EEE 143: Switching Theory and Digital Logic Design

Lecture 15: Towards Programmable Circuits,
Course wrap-up

1

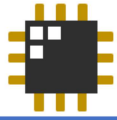


Recall: Factorial ASM



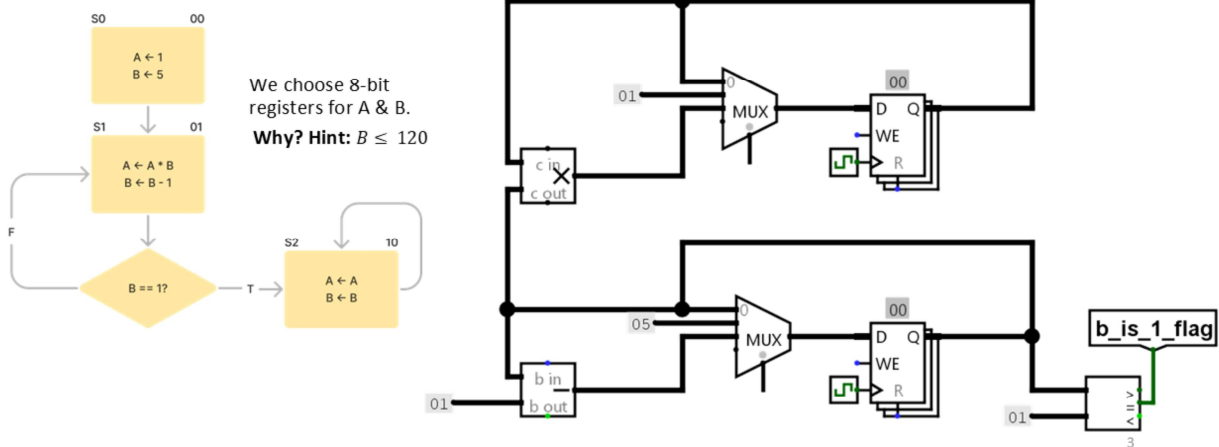
2

The ASM diagram and the clock/timing diagram together demonstrate how the algorithm is implemented in hardware through state transitions and register transfers synchronized to the clock. The ASM shows the logical flow of states, decisions, and data operations, while the timing diagram reveals the precise moments when register values update at each clock's positive edge. This highlights the critical cycle-sensitive nature of ASMs, emphasizing that operations scheduled in one state only take effect after the clock edge, which can lead to extra or delayed updates if not carefully designed. Understanding this interplay between ASM flow and clock timing is essential to correctly implement and debug digital circuits based on algorithmic state machines.



Recall: Factorial datapath

The **datapath** contains *what* we need to do to the registers.
Hence, we have a multiplier, a subtractor, constants, and MUXes.

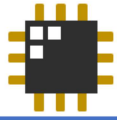


The datapath is responsible for carrying out the actual operations described in the ASM, so it includes all the necessary components to manipulate the registers involved.

In this case, we need to compute the factorial by repeatedly multiplying two values and decrementing one of them. Therefore, our datapath must contain a **multiplier** (to compute $A \times B$), a **subtractor** (to compute $B - 1$), and **constant sources** such as 1 (for decrementing) and 0 (for initialization). We also include **multiplexers (MUXes)** to control which values are loaded into the registers depending on the state of the system.

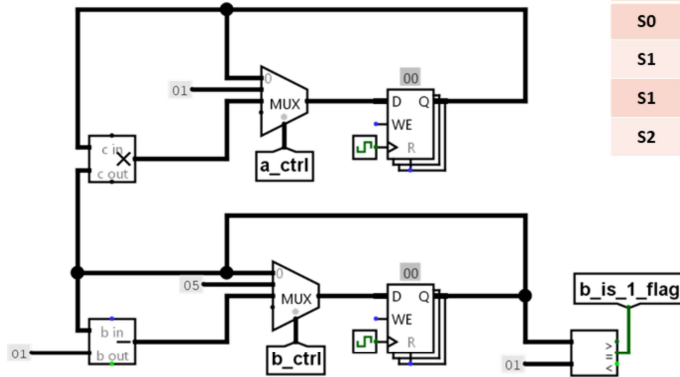
For storage, we use **8-bit registers** for both A and B. This is a practical choice because we know that the maximum value we expect for the factorial operation is 120 (i.e., $5! = 120$). Since 8-bit registers can represent values up to 255, they are more than sufficient to store any intermediate or final result in our factorial computation.

Overall, this datapath reflects the "what" in the ASM: the actual arithmetic and data movement operations needed to compute the factorial.



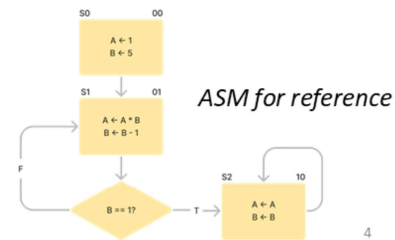
Recall: Factorial datapath controller

We still need to tell the datapath *when* to do what things, and this is where **control** comes in. *Control* contains the states! It importantly gives the **MUX SELECTS**.



Let's describe it using a state table!

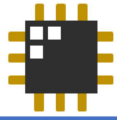
State	b_is_1	Next State	a_ctrl	b_ctrl
S0	x	S1	01 (choose 1)	01 (choose 5)
S1	0	S1	10 (choose A*B)	10 (choose B-1)
S1	1	S2	10 (choose A*B)	10 (choose B-1)
S2	x	S2	00 (choose A)	00 (choose B)



In our factorial example, the **datapath** contains all the elements necessary to perform operations—registers, multipliers, subtractors, and multiplexers (MUXes). However, the datapath **doesn't decide on its own when to do what**. That's where the **control** component comes into play.

The **control unit** is responsible for sequencing and decision-making. It holds the **states** we defined in our ASM (Algorithmic State Machine), and it ensures that at each clock cycle, the correct operations are triggered. Most importantly, it **determines the select lines for the multiplexers** in the datapath. These **MUX SELECTS** are what enable the datapath to pick between different inputs at the right time—for example, whether A should load a constant 1, retain its value, or compute $A \times B$.

In short, while the datapath does the actual work, the **control tells it what to do and when**, making sure everything runs according to the ASM we've designed.

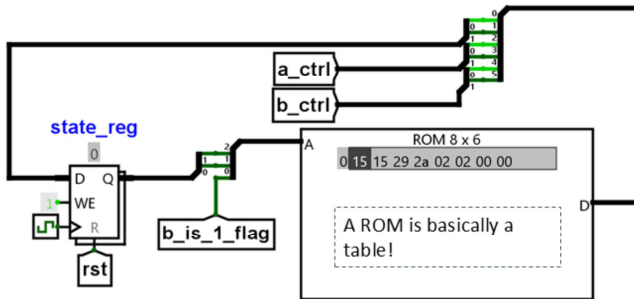


Recall: Factorial datapath controller

We can do the **usual synthesis** from here to make the controller and connect it to the datapath. However, an easy trick to make a controller is by using a **read-only memory (ROM)** for the combinational part:

ROM outputs = {a_ctrl, b_ctrl, next_state} ← 6 bits

ROM inputs = {state, b_is_0} ← 3 bits



State	b_is_0	Next State	a_ctrl	b_ctrl
S0	x	S1	01 (choose 1)	01 (choose 5)
S1	0	S1	10 (choose A*B)	10 (choose B-1)
S1	1	S2	10 (choose A*B)	10 (choose B-1)
S2	x	S2	00 (choose A)	00 (choose B)

ROM_in	ROM_out (bin)	ROM_out (hex)
00 x	01 01 01	0x15
01 0	10 10 01	0x29
01 1	10 10 10	0x2A
10 x	00 00 10	0x02

Note: ROM input and output bit order can change depending on how the circuit was wired

5

The next step is to **synthesize** the actual hardware that implements it. Normally, this involves creating logic circuits for the next-state logic and control signals. However, there's a clever and **simpler alternative** for implementing the **combinational part** of the controller: we can use a **Read-Only Memory (ROM)**.

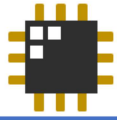
In this method, we **encode all the control outputs and next state transitions** as data entries in the ROM. The **inputs to the ROM** are the **current state** and any **condition flags** (in our case, the b_is_0 flag from the comparator). So, we use **3 input bits**: two for the state (since we only have 3 states, 2 bits are enough) and one for the b_is_1 condition. This gives us a total of $2^3 = 8$ possible input combinations.

The **outputs of the ROM** are the control signals needed for the datapath and the encoded **next state**. Specifically, we need:

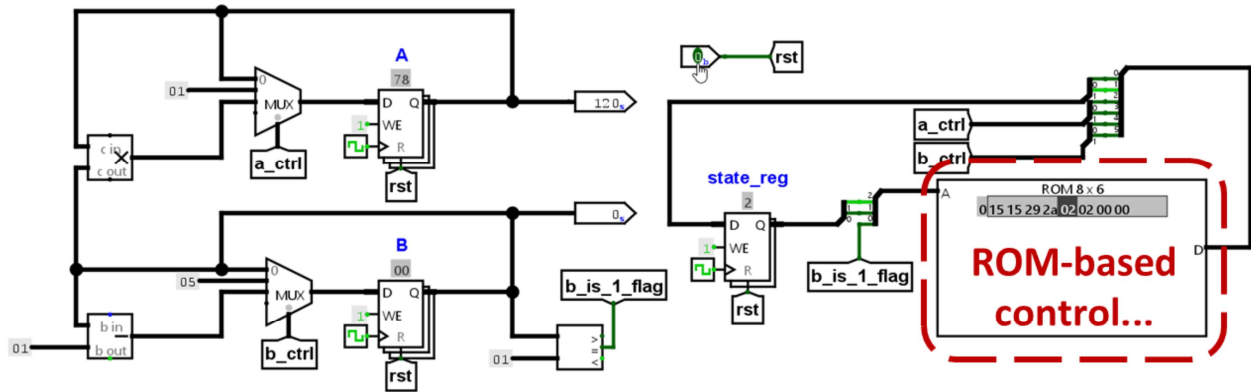
- next_state (2 bits to indicate which state to move to)
- a_ctrl (2 bits for A's MUX select), and
- b_ctrl (2 bits for B's MUX select).

That gives us a total of **6 output bits** per ROM entry.

This setup allows us to **store all possible control decisions** in a compact table inside the ROM, which behaves like a lookup table. For any given state and condition flag, the ROM instantly outputs the appropriate MUX controls and next state. This is a fast and reliable way to implement small controllers without having to manually design combinational logic gates for each transition.



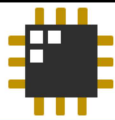
Factorial complete circuit



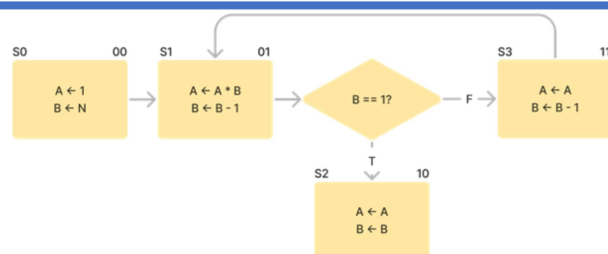
We implemented control with a ROM... which means maybe we can change what this thing does!

6

We implemented control using a ROM... which means there's something interesting we can do with it. Since the ROM determines the behavior of the controller, changing the contents of the ROM essentially changes what the entire system does—without rewiring or redesigning the hardware. This opens up a powerful idea: **what if we could reprogram the behavior of a circuit just by updating the ROM?** This hints at the concept of more general-purpose or programmable hardware—a step closer to how processors and microcontrollers work.



Factorial every two numbers



State	b_is_1	Next State	a_ctrl	b_ctrl
S0	x	S1	01 (choose 1)	01 (choose 5)
S1	0	S3	10 (choose A*B)	10 (choose B-1)
S1	1	S2	10 (choose A*B)	10 (choose B-1)
S2	x	S2	00 (choose A)	00 (choose B)
S3	x	S1	00 (choose A)	10 (choose B-1)

This new ASM performs 'B-1' twice before the multiplication, making it 'skip' every other number

Recall:

ROM outputs = {a_ctrl, b_ctrl, next_state} ← 6 bits

ROM inputs = {state, b_is_0} ← 3 bits

ROM_in	ROM_out (bin)	ROM_out (hex)
00 x	01 01 01	0x15
01 0	10 10 11	0x2B
01 1	10 10 10	0x2A
10 x	00 00 10	0x02
11 x	00 10 01	0x09

7

This modified circuit now computes the **factorial of every second number**, essentially calculating something like:

$$A = n \times (n-2) \times (n-4) \times \dots$$

To do this, the ASM is updated to insert an extra subtraction before each multiplication, effectively skipping every other number in the factorial computation.

Key Changes in ASM

To enable this behavior, a **new state S3** is introduced. Here's what changes:

- In **state S1**, when b_is_1 is 0 (i.e., $B \neq 1$), the next state becomes **S3** instead of looping back to S1.
- **State S3** performs an extra subtraction: B is decremented again ($B \leftarrow B - 1$), and A remains unchanged ($A \leftarrow A$). This makes the system "skip" every second value of B.
- From S3, the system always transitions back to S1, regardless of b_is_1.

Datapath Behavior in S3

In state S3:

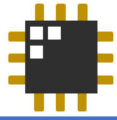
- A input = **previous A** → control value: 00
- B input = **B - 1** → control value: 10

ROM Table Updates

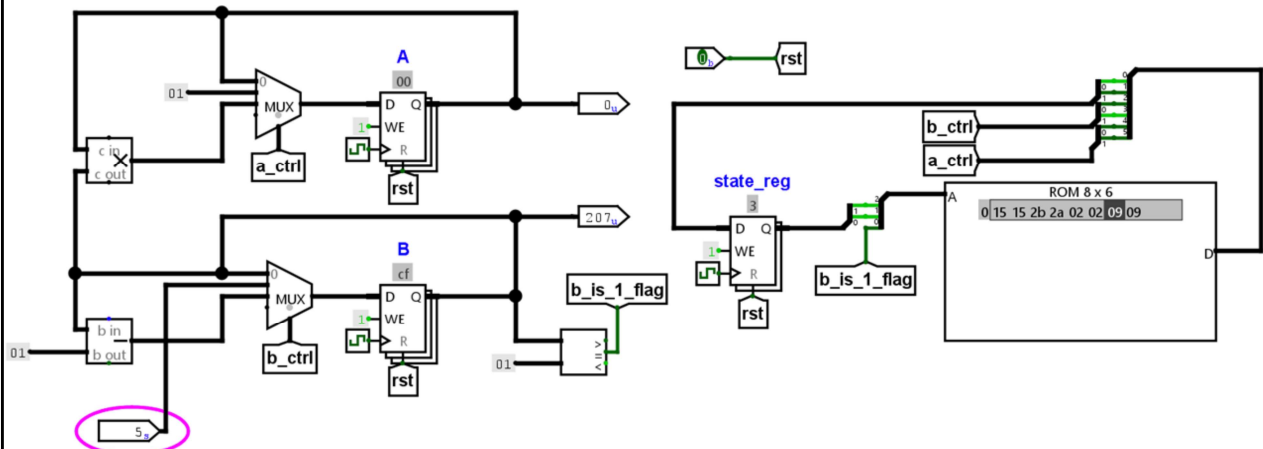
The ROM now includes an additional input-output mapping to support S3.

- 01 0: When in state S1 and b_is_1 = 0, move to S3 (11), doing AxB and B-1.
- 11 x: In state S3, regardless of b_is_1, go back to S1 (01), with A unchanged and B-1.

This design showcases how **modifying the controller logic and expanding the ASM** allows us to define entirely new computational behaviors using the same datapath structure—just by updating control states and ROM contents.



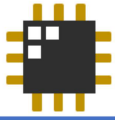
Factorial every two numbers



This doesn't work yet with even numbers— can you guess why? Can you fix it?

8

This slide shows the final circuit implementation for the modified factorial calculator—one that skips every other number in the sequence. The architecture is very similar to the original factorial circuit, with the same datapath components: registers for A and B, a multiplier, a subtractor, and multiplexers that determine what values to load into each register. The control unit, implemented using a ROM, has been updated to include an additional state, S3, allowing the machine to perform two decrements on B for every one multiplication on A. This effectively creates a loop that calculates something like $A = B \times (B-2) \times (B-4) \dots$, as discussed earlier.



Motivating programmable circuits

- We can use the **same datapath** with a **different ROM controller** to implement different functionalities!
- What happens if we make the datapath *very general*?

***It'll be able to do many different things
when you replace the controller!***

9

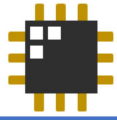
We are now introducing an exciting direction in digital design: the concept of **programmable circuits**. It builds on what we've already done—using the same **datapath** but swapping out the **ROM controller** to change behavior. For instance, we previously implemented a factorial calculator, then modified the ROM to compute factorials of every other number—all without altering the datapath.

So now, we ask: *What happens if we make the datapath even more general?* Instead of tailoring the datapath to a specific function like multiplication or subtraction, we design it to support a **broad set of basic operations**—like addition, logical operations, comparisons, and flexible data routing. In that case, the **same hardware** can be reused across many different functionalities, depending on what the controller (or program) tells it to do.

This idea leads us directly toward the concept of **programmability** in hardware. The datapath becomes a general-purpose "engine," and the controller becomes the "software" or "program" that tells it how to behave. This is essentially how microcontrollers and CPUs work: by keeping a fixed datapath and loading different instructions into a control unit or memory to execute

different tasks.

It's a powerful idea because it means we don't have to build new hardware for every problem—we just need to make the hardware flexible enough to adapt to many problems via control. This flexibility is what makes programmable digital systems so versatile and efficient in real-world applications.

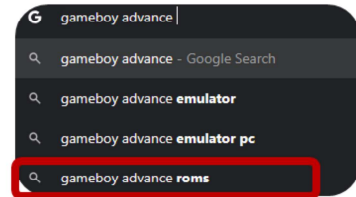


Motivating programmable circuits

- We can use the **same datapath** with a **different ROM controller** to **implement different functionalities!**
- What happens if we make the datapath *very general*?

*It'll be able to do many different things
when you replace the controller!*

- Different ROM controllers are exactly what these are

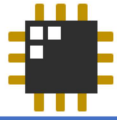


10

Let's consider **Game Boy Advance (GBA) ROMs** as a real-world example to illustrate the concept of different ROM controllers controlling the same underlying hardware. Each GBA game cartridge contains a ROM that acts as the controller, directing the hardware on what operations to perform.

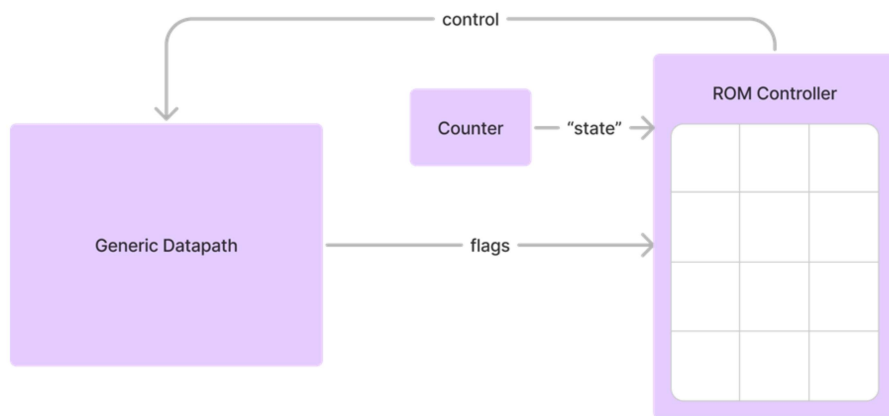
Just like how we discussed swapping ROM controllers to change the behavior of the same datapath in our earlier examples, GBA cartridges use different game ROMs to make the same handheld console run completely different games. The hardware—the CPU, graphics, sound, and input devices—remains the same, but the ROM code determines the program's functionality.

This highlights a key advantage of programmable systems: **reusability of hardware with different ROM controllers (programs)** that define new functionalities without needing to redesign or rebuild the physical circuit. So, different ROM controllers are exactly what these game cartridges are—software-like control codes that direct the same datapath hardware to behave differently, making each game unique.



Programmable circuits

Maybe something like this would work!



11

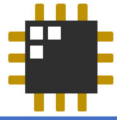
Here, we can see a conceptual diagram of a **generic datapath-controller system** that could handle a variety of tasks through programmable control. The datapath represents the core hardware—registers, arithmetic units, multiplexers, and so on—where the actual data operations happen.

An arrow labeled **“flags”** points from the datapath to the ROM controller, indicating that the datapath sends status signals or condition flags (like zero, carry, or comparison results) back to the controller. These flags inform the controller about the current state of the datapath’s operations, which helps decide the next steps.

There’s also a **counter** connected to the ROM controller, with an arrow labeled **“state”** going into the controller. This counter keeps track of the current state number, enabling the ROM controller to fetch the appropriate control signals based on that state.

From the ROM controller, an arrow labeled **“control”** points back to the datapath. This means the controller sends signals to dictate what operations the datapath should perform next, such as selecting which registers to use,

enabling arithmetic units, or controlling multiplexers.

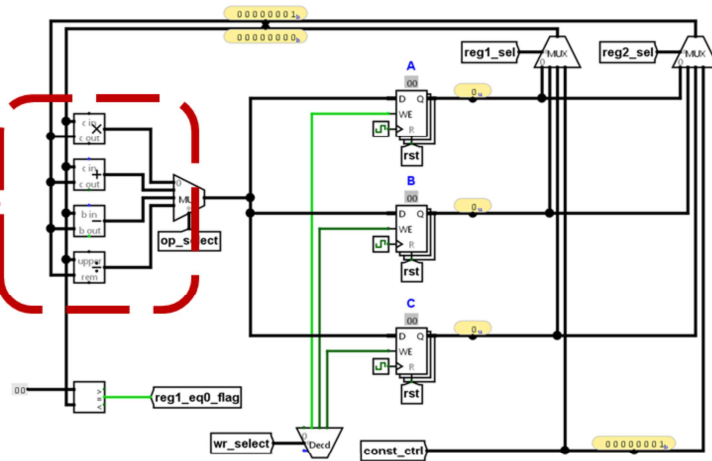


Towards programmable circuits

Let's make the factorial
datapath more general!

Support more operations
other than multiply

op_select would select
which operation to use

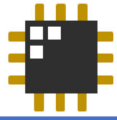


12

Let's consider making the factorial datapath more versatile by expanding its capabilities beyond just multiplication. Instead of being limited to one fixed operation, the datapath can support multiple arithmetic or logical functions, such as addition, subtraction, shifting, or even AND/OR operations.

To control which operation is performed at any given time, we introduce a control signal called **op_select**. This signal acts like a selector that tells the datapath which specific operation to carry out on the inputs or registers.

By adding this flexibility, the datapath becomes a more general-purpose unit, capable of handling a wider range of tasks. This not only makes it reusable for different algorithms or applications but also paves the way toward more programmable and adaptable hardware designs.

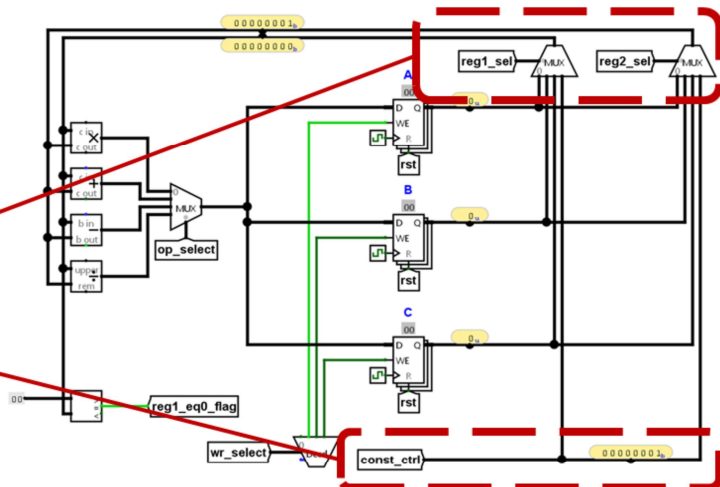


Towards programmable circuits

Let's make the factorial datapath more general!

Choose operand 1 and operand 2

which can be a register
(controlled by *reg1_sel*, *reg2_sel*)
or a constant from the controller
(*const_ctrl*)

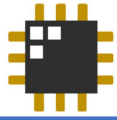


15

In the datapath, we often need to select two operands for an operation, such as addition or multiplication. These operands can come from different sources: they might be the values stored in registers, or they could be constant values provided directly by the controller.

To choose which registers supply these operands, we use control signals like **reg1_sel** and **reg2_sel**. These signals select which registers' contents become operand 1 and operand 2, respectively. Alternatively, the controller can supply a constant value instead, controlled by **const_ctrl**, which overrides the register inputs when needed.

This flexibility in operand selection allows the datapath to perform a wide variety of operations using different inputs, making it more versatile and powerful.



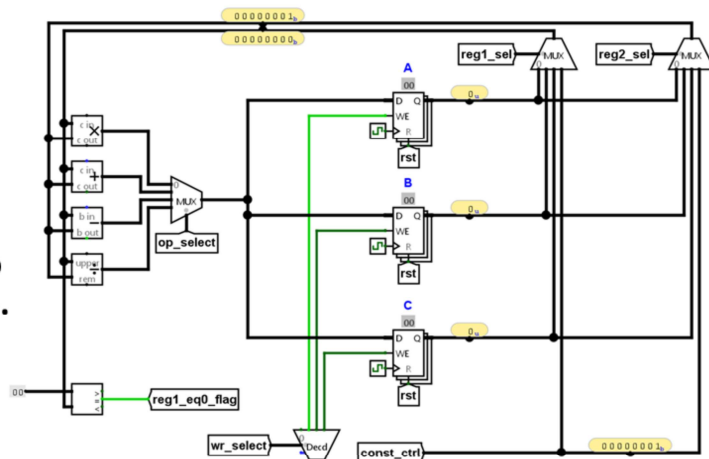
“Programming language”

What do we put into the controller ROM?

Register Transfers!

With this datapath, we can do one register transfer per cycle.

Maybe we can call this “programming”

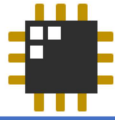


16

What goes into the controller ROM? Essentially, it stores the **register transfers**—the specific instructions that tell the datapath what operations to perform on the registers each clock cycle.

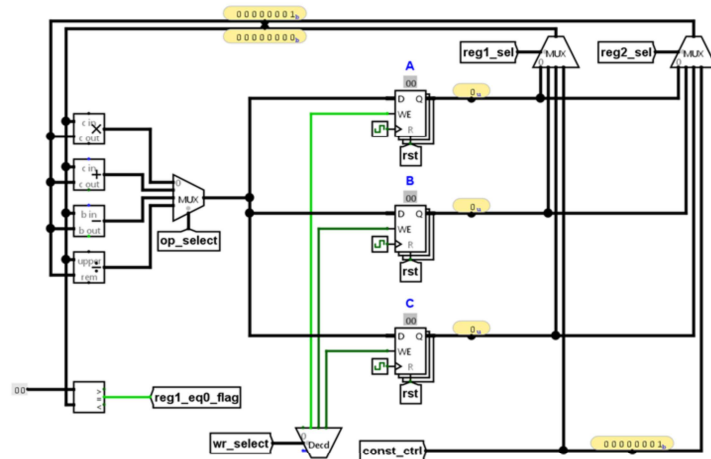
With the current datapath design, we’re able to execute one register transfer per cycle. This means every clock tick, the controller decides exactly which register operation happens next, directing the flow of data and computation.

This approach is very powerful because it lets us think of the controller as a kind of “program.” By changing the ROM contents—the sequence of register transfers—we effectively **program** the hardware to perform different tasks without changing the physical datapath. This is the foundation of programmable circuits.



“Programming language”

```
1 // Factorial of 5
2 RA <- 1
3 RB <- 5
4 RA <- RA * RB
5 RB <- RB - 1
6 RA <- RA * RB
7 RB <- RB - 1
8 RA <- RA * RB
9 RB <- RB - 1
10 RA <- RA * RB
11 RB <- RB - 1
12 RA <- RA * RB
13 RB <- RB - 1
```

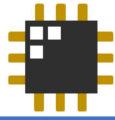


17

Building on the idea of “programming” with register transfers in the controller ROM, here’s a concrete example using the factorial of 5. The sequence of register transfers shows how the datapath operations are organized step-by-step, like instructions in a program.

First, we initialize register A (RA) to 1 and register B (RB) to 5. Then, we repeatedly multiply RA by RB, and decrement RB by 1 each cycle. This sequence continues until RB reaches 1, effectively calculating 5! through successive multiplications.

This sequence for factorial demonstrates how hardware can be *programmed* to carry out complex tasks step-by-step, showing the principle behind more advanced, flexible digital systems.



“Programming language”

```

1 // Factorial of 5
2 RA <- 1
3 RB <- 5
4 RA <- RA * RB
5 RB <- RB - 1
6 RA <- RA * RB
7 RB <- RB - 1
8 RA <- RA * RB
9 RB <- RB - 1
10 RA <- RA * RB
11 RB <- RB - 1
12 RA <- RA * RB
13 RB <- RB - 1

```

// Factorial of 5	const_ctrl (hex)	reg1_sel	reg2_sel	op_select	wr_select	Hex
RA <- RA + 1	01	00 (RA)	11 (const)	01 (+)	01 (A)	0135
RB <- RB + 5	05	01 (RB)	11 (const)	01 (+)	10 (B)	0576
RA <- RA * RB	x	00 (RA)	01 (RB)	00 (*)	01 (A)	xx11
RB <- RB - 1	01	01 (RB)	11 (const)	10 (-)	10 (B)	017A
RA <- RA * RB	x	00 (RA)	01 (RB)	00 (*)	01 (A)	xx11
RB <- RB - 1	01	01 (RB)	11 (const)	10 (-)	10 (B)	017A
RA <- RA * RB	x	00 (RA)	01 (RB)	00 (*)	01 (A)	xx11
RB <- RB - 1	01	01 (RB)	11 (const)	10 (-)	10 (B)	017A
RA <- RA * RB	x	00 (RA)	01 (RB)	00 (*)	01 (A)	xx11
RB <- RB - 1	01	01 (RB)	11 (const)	10 (-)	10 (B)	017A
RA <- RA * RB	x	00 (RA)	01 (RB)	00 (*)	01 (A)	xx11
RB <- RB - 1	01	01 (RB)	11 (const)	10 (-)	10 (B)	017A

Check the datapath for proper assignment of the select signals!

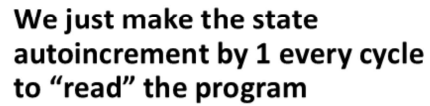
Let’s break down the factorial program into detailed control signals needed to drive the datapath each clock cycle. Essentially, it shows how each instruction in the sequence is translated into hardware control inputs.

The table lists columns for every step of the program, specifying:

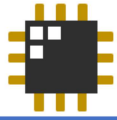
- **const_ctrl**: The constant value fed as an operand when needed (like 1 or 5).
- **reg1_sel** and **reg2_sel**: Which registers are selected as operand 1 and operand 2 for the operation.
- **op_select**: Which operation to perform (e.g., multiply or subtract).
- **wr_select**: Which register is chosen to be written or updated in that cycle.
- **Hex**: The combined control signals converted into a hexadecimal value representing the ROM output for that instruction.

This layout shows exactly how the control ROM encodes each instruction’s control signals to direct the datapath properly. By doing so, it illustrates the core idea of programmable hardware: the “program” is a sequence of control words that dictate how data moves and changes inside the circuit.

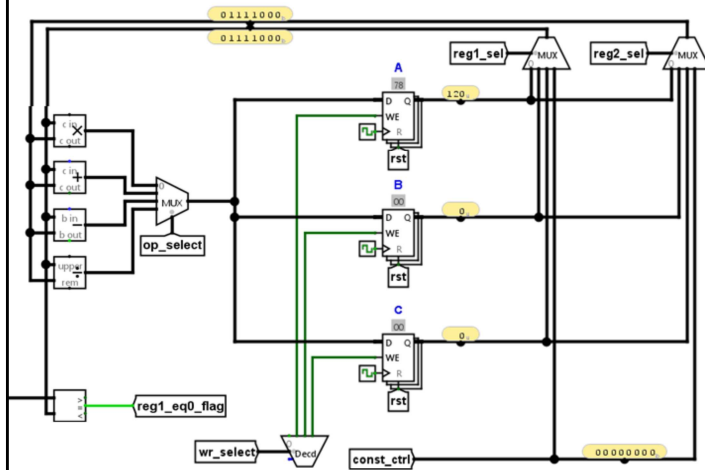
Program of $A = 5!$



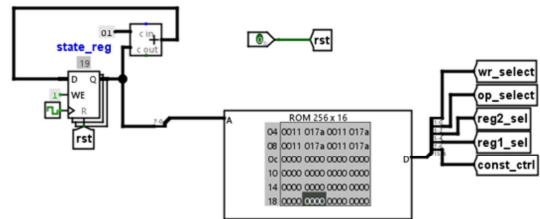
This highlights how the control ROM combined with an automatic state counter lets the hardware behave like a simple processor executing a program, bridging the gap between algorithm and physical circuit. It shows a clear transition from fixed-function logic to programmable hardware by treating control states as instruction addresses.



Program of $A = 5!$

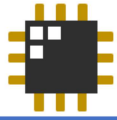


With different ROMs, we can do Fibonacci, Factorial, and Triangular Numbers with this datapath!



20

With this flexible datapath design, simply changing the contents of the control ROM lets us implement entirely different functions — like Fibonacci, Factorial, or Triangular numbers — without altering the hardware itself. The ROM acts like a program memory, and by loading different “programs” (control sequences), the same circuit can perform various complex calculations. This demonstrates the power of programmable hardware: the datapath provides the functional building blocks, while the ROM controller directs the sequence of operations. So, the hardware stays the same, but the behavior changes entirely depending on what’s stored in the ROM.



Making shorter programs

too repetitive!

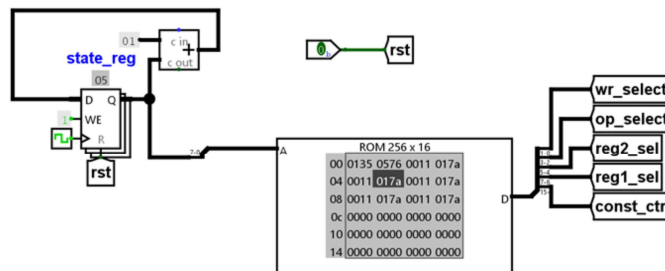
```
1 // Factorial of 5
2 RA <- 1
3 RB <- 5
4 RA <- RA * RB
5 RB <- RB - 1
6 RA <- RA * RB
7 RB <- RB - 1
8 RA <- RA * RB
9 RB <- RB - 1
10 RA <- RA * RB
11 RB <- RB - 1
12 RA <- RA * RB
13 RB <- RB - 1
```

maybe this is better?

```
1 // Factorial of 5
2 RA <- RA + 1
3 RB <- RB + 5
4 RA <- RA * RB
5 RB <- RB - 1
6 if (RB != 0) state_reg - 2
```

However, we repeated a lot of commands here.

Maybe we can implement an **if/else** on the controller...



How do we change this so it can 'loop'?

21

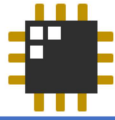
In the previous program, many commands were repeated, making it longer and less efficient. To improve this, we could implement conditional logic—like an if/else statement—directly in the controller. This would allow the program to 'loop,' repeating certain operations only as needed instead of writing out every single step. For example, the revised sequence increments RA and RB, multiplies RA by RB, decrements RB, and then checks if RB is not zero. If it isn't, the program jumps back to an earlier state (state_reg - 2) to repeat the loop. This approach makes the program shorter, more flexible, and closer to how software loops work, improving efficiency in hardware control.



Adding a constant to the state register is called “BRANCHING”
(We ‘branch out’ to different parts of the program)



Control flow in hardware programming involves more than just stepping through instructions one by one. Instead of always incrementing the state register by 1, we can add a constant value to it, allowing the program to jump—or branch—to different parts of the instruction sequence. This branching capability lets the controller move to a specific state or address in the ROM rather than following a linear path. By adding this flexibility, the hardware can implement loops, conditionals, or skip over certain instructions, similar to how software uses branches and jumps to control execution flow. This makes the controller much more powerful and adaptable.

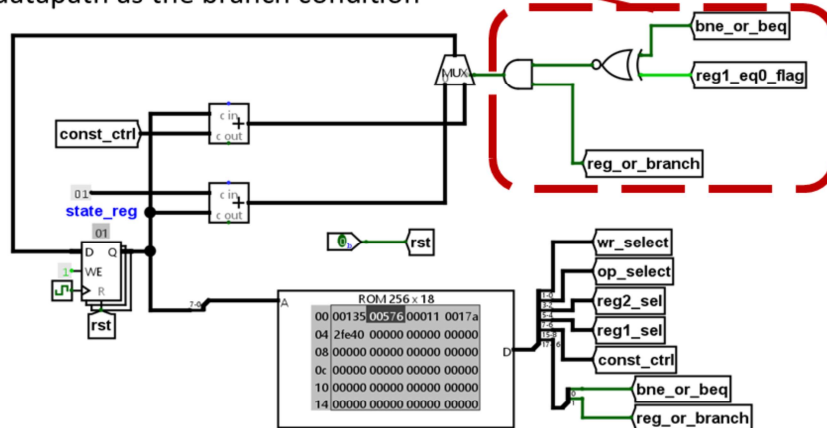


Control flow

New control outputs: **'branch type'**

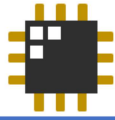
We use the comparison flag from the datapath as the branch condition

Branch types:
Not Equal (NE)
Equal (EQ)



23

New control outputs called 'branch type' enhances the controller's decision-making ability. These branch types determine whether the program should jump to a different state based on conditions evaluated in the datapath. Specifically, the datapath provides a comparison flag—often indicating if a register value is zero or not—which acts as the branch condition. The two common branch types are Not Equal (NE) and Equal (EQ). If the condition matches the branch type (for example, if the flag indicates "not equal" and the branch type is NE), the controller performs the branch by adjusting the state register accordingly. This mechanism enables conditional branching, allowing the hardware program to implement if-else logic and loops efficiently.



Factorial with branching

```

1 // Factorial of 5
2 RA <- RA + 1
3 RB <- RB + 5
4 RA <- RA * RB
5 RB <- RB - 1
6 if (RB != 0) state_reg - 2

```

// Factorial of 5	reg_or_branch	bne_or_beq	const_ctrl (hex)	reg1_sel	reg2_sel	op_select	wr_select	Hex
RA <- RA + 1	0	x	01	00 (RA)	11 (const)	01 (+)	01 (A)	0 0135
RB <- RB + 5	0	x	05	01 (RB)	11 (const)	01 (+)	10 (B)	0 0576
RA <- RA * RB	0	x	x	00 (RA)	01 (RB)	00 (*)	01 (A)	0 xx11
RB <- RB - 1	0	x	01	01 (RB)	11 (const)	10 (-)	10 (B)	0 017A
if (RB != 0) state_reg - 2	1	0 (bne)	FE (-2)	01 (RB)	00	00	00 (none)	2 fe40

24

Let's break down the factorial-of-5 program with branching into detailed control signals that the hardware controller uses to execute each step. Each row corresponds to a register transfer or operation in the program, showing precisely how the datapath is controlled at every clock cycle.

The **register transfers** column lists the operations happening in that step, such as loading constants, performing multiplication, or decrementing the register.

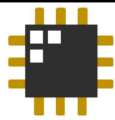
The **reg_or_branch** signal indicates whether the controller is performing a normal register operation or a branch instruction—essentially whether it's advancing sequentially or conditionally jumping to another state.

The **bne_or_beq** field specifies the branch condition type: either branch if not equal (BNE) or branch if equal (BEQ). This decides when the controller will alter the program flow based on the datapath's comparison flags.

The **const_ctrl**, **reg1_sel**, **reg2_sel**, **op_select**, and **wr_select** columns define the multiplexers and ALU control signals that select constants or registers as operands, choose the operation to perform (like multiplication or

subtraction), and determine which register gets updated.

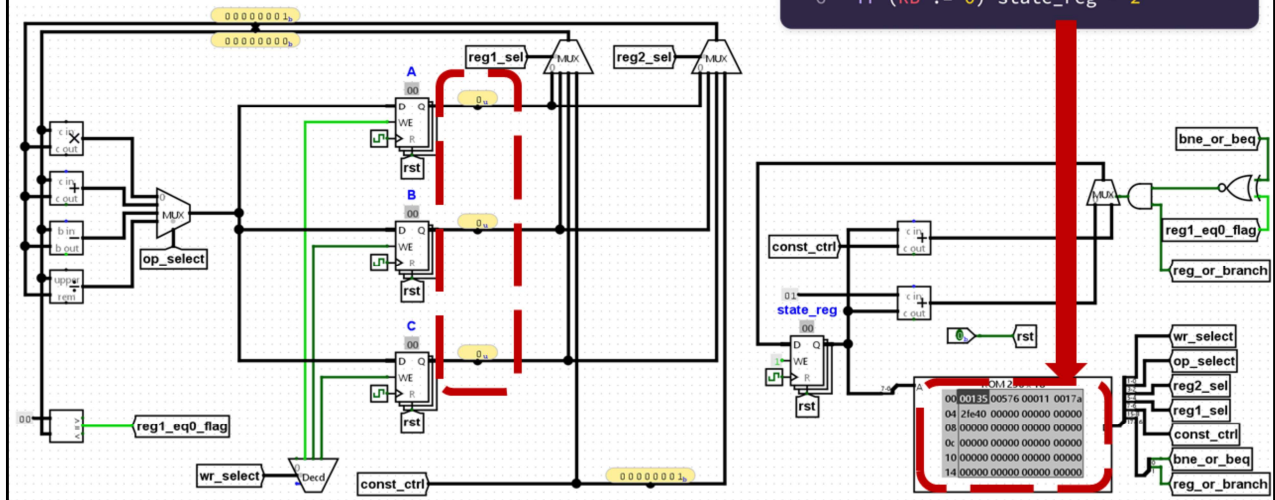
Finally, the **Hex** column shows the concatenated binary control signals in hexadecimal form—this is the actual data stored in the ROM controller that drives the hardware behavior.



Factorial with branching

This circuit performs the factorial calculation

```
1 // Factorial of 5
2 RA <- RA + 1
3 RB <- RB + 5
4 RA <- RA * RB
5 RB <- RB - 1
6 if (RB != 0) state_reg - 2
```



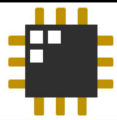
Here's the final circuit for the factorial of 5 with branching, which illustrates how we have integrated conditional control flow into the datapath-controller system. This circuit doesn't just execute a fixed sequence of operations anymore; it can make decisions based on the value of the registers—specifically, it uses branching to loop back or proceed depending on a condition.

In this design, the controller now includes branch logic that checks the comparison flags from the datapath (like whether register B is zero). Based on this flag and the branch type (branch if not equal or branch if equal), the controller decides whether to continue looping (repeating multiplication and decrement steps) or to stop and finalize the result.

This branching mechanism enables the circuit to efficiently handle loops, such as repeatedly multiplying while decrementing B, without manually listing every single operation step. It effectively makes the hardware "program" able to handle iterations, similar to how software loops work.

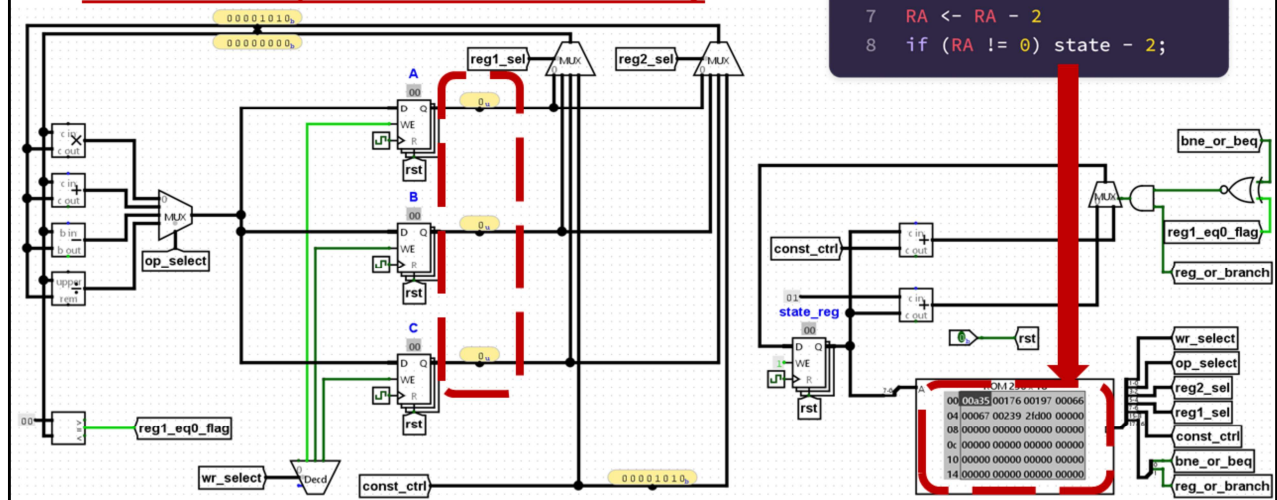
The inclusion of the branch logic in the controller alongside the datapath

completes the factorial calculation circuit, making it more flexible and scalable for various inputs, not just fixed constants. This circuit can now calculate factorials by iterating through the loop controlled by the branch condition, showing a crucial step towards programmable hardware behavior.



Fibonacci with branching

This circuit performs the Fibonacci calculation
Same circuit, different functionality!



Let's look how the same datapath-controller hardware, with branching capabilities, can be reprogrammed to calculate Fibonacci numbers instead of factorials — showcasing the power of programmable circuits.

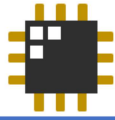
The circuit performs the Fibonacci sequence using the registers RA, RB, and RC, with a series of register transfers that update these registers step-by-step:

- RA is incremented by 10, serving as a counter or control value.
- RB and RC are incremented by 1 each.
- Then, RB and RC are updated by adding each other ($RB \leftarrow RB + RC$, $RC \leftarrow RC + RB$), which captures the core addition pattern of Fibonacci.
- RA is decremented by 2 to control the loop count.

The key control flow is the conditional branch: if RA is not zero, the state counter is decremented by 2 ($state - 2$), effectively looping back two steps in the program to repeat the process.

This example highlights how the same hardware circuit, simply by loading a different ROM with new control instructions, can perform an entirely different computation. The branching logic allows the circuit to loop dynamically, enabling complex iterative processes like Fibonacci calculations

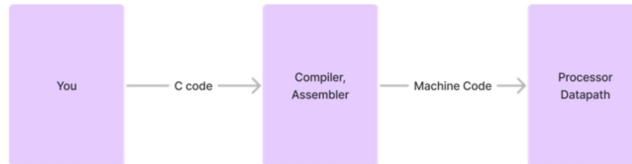
without changing the physical hardware—only the controller ROM changes. This flexibility is what makes programmable hardware so powerful.



The hardware/software interface

It is difficult and error-prone to translate register transfers into HEX for the memory.

In modern computers, this process is automated by **compilers and assemblers**.



```
hello.c (-My Documents\Programming Projects) - VM
#include <stdio.h>
int main() {
    printf("Hello, World!");
    return 0;
}

This is easy!

"hello.c" [null] 5L, 79C      1,1      011
```



```
C:\WINDOWS\system32\cmd.exe - gdb hello.exe
Dump of assembler code for function main:
0x00401460 <+0>: push    %ebp
0x00401461 <+1>: mov     %esp,%ebp
0x00401463 <+3>: and     $0xffffffff,%esp
0x00401466 <+6>: sub     $0x10,%esp
0x00401469 <+9>: call    0x00401460 @.main
0x0040146e <+14>: movl    $0x405064,%esp
0x00401475 <+21>: call    0x00401460 @.printf
0x0040147a <+26>: mov     %ebp,%eax
0x0040147f <+31>: leave   %ebp
0x00401480 <+32>: ret
0x00401481 <+33>: nop
0x00401482 <+34>: nop
0x00401483 <+35>: nop
0x00401484 <+36>: xchgb   %al,%ah
0x00401486 <+38>: xchgb   %ah,%al
0x00401488 <+40>: xchgb   %al,%ah
0x0040148a <+42>: xchgb   %ah,%al
0x0040148c <+44>: xchgb   %al,%ah
0x0040148e <+46>: xchgb   %ah,%al
End of assembler dump.
(gdb)

WHAT!???
```

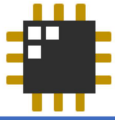
Let's talk about the crucial interface between human-readable instructions and the actual hardware execution—what we call the hardware/software interface.

Manually translating register transfers or instructions into the exact hexadecimal codes needed for the ROM or memory is difficult and prone to errors. Writing machine code by hand is tedious, and even small mistakes can cause big problems in the circuit's behavior.

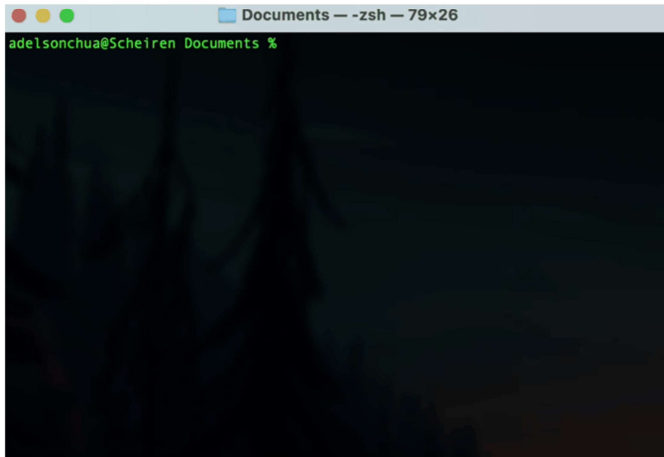
To solve this, modern computer systems use automated tools like compilers and assemblers. These tools take high-level human-readable code—like C language—and convert it step-by-step into machine code (hexadecimal instructions) that the processor's datapath can understand and execute.

The photos show example of this transformation: one with human-readable source code and the other showing the resulting sequence of hex values. This automation bridges the gap between programmers and hardware, making programming faster, more reliable, and accessible.

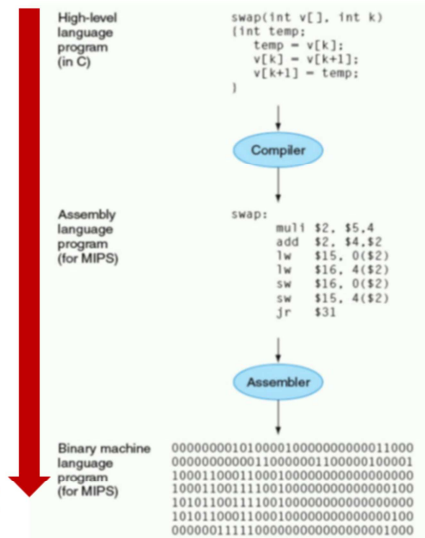
So essentially, today's complex systems rely on this chain: You write code → Compiler/Assembler converts it → Machine code is loaded into hardware → Processor executes the instructions seamlessly. This process is fundamental to making programmable hardware practical and scalable.



The hardware/software interface



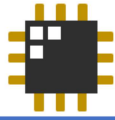
Conversion from C code to Assembly to Hex/Binary



Let's look at the demonstration of the process of converting a simple C program into assembly code and then into machine code (hex/binary), all done through a command-line interface.

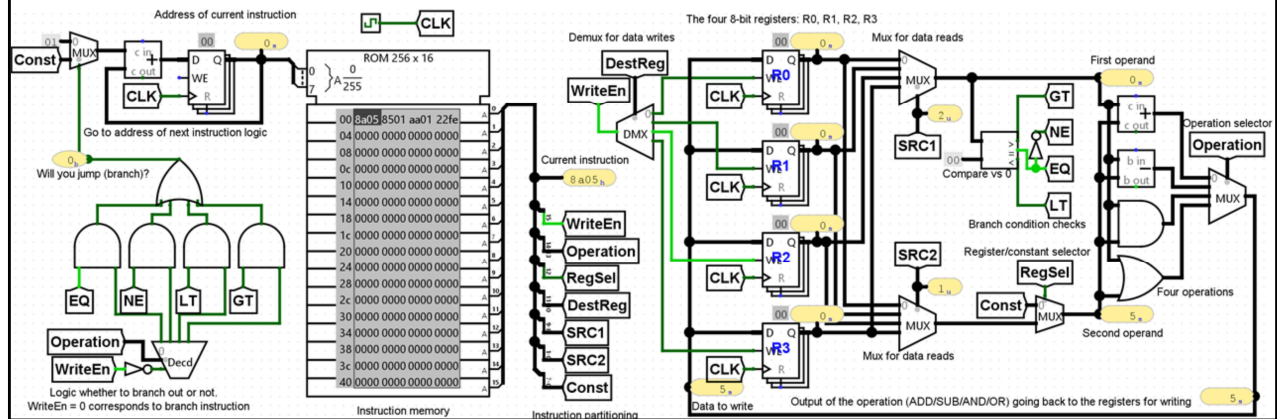
It starts by opening a C source file (like test.c) in a text editor, such as Vim, showing the human-readable C code. Then, using compiler commands right in the terminal, it compiles this C code into assembly language—an intermediate, more hardware-near representation that's still somewhat readable to humans.

Next, the assembly code is further processed by the assembler to generate machine code, which is the actual binary or hex instructions that the processor's datapath executes.



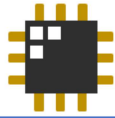
Ultra-generic datapaths: Processors

The EEE 113 Architecture Design that's Easy to Learn (ADEL) processor

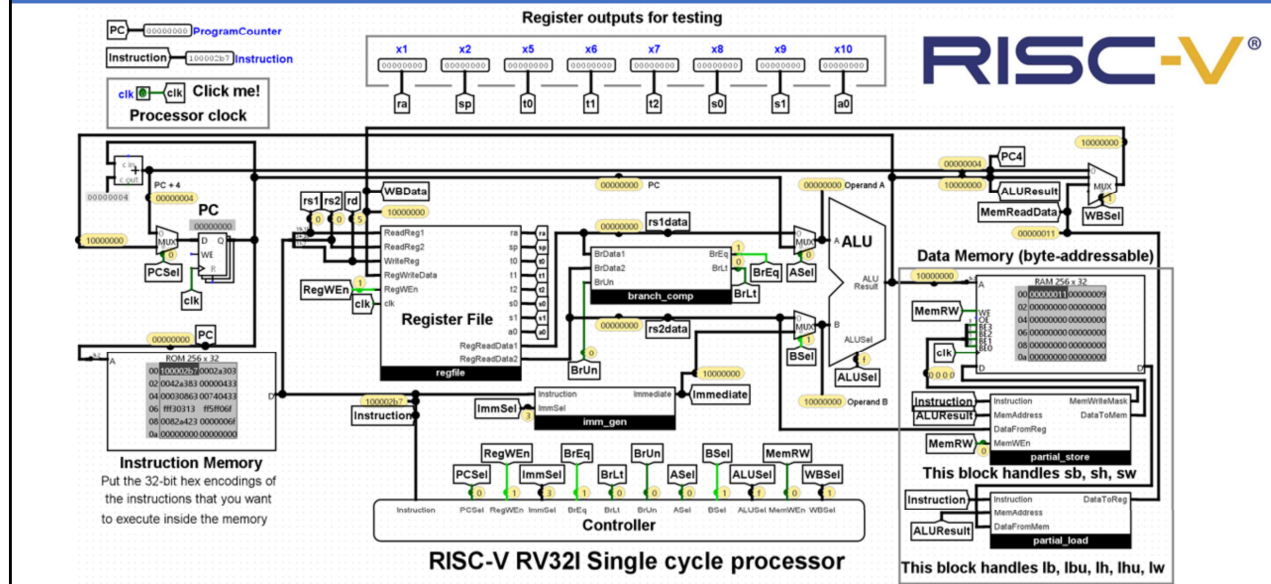


Ultra-generic datapaths are what we find in processors—the heart of modern computing devices. Unlike specialized datapaths designed for a single function or a small set of tasks, ultra-generic datapaths are built to handle a vast range of operations and instructions. They support complex instruction sets, multiple registers, various arithmetic and logic operations, memory access, and control flow mechanisms like branching and looping.

This extreme flexibility allows processors to run countless different programs, from simple calculations to complex applications, simply by loading different software. Essentially, processors are the ultimate example of programmable hardware, where the datapath is designed to be as general as possible, and all functionality is determined by the instructions fed into the system. This makes processors highly versatile and essential for modern computing.



To EEE 153: RISC-V Processor



RISC-V is an open-standard, simple, and flexible instruction set architecture (ISA) that's gaining wide popularity in both academia and industry. It's designed to be easy to implement, scalable, and extensible, making it ideal for a variety of applications—from small embedded devices to powerful processors. In the next course, we'll explore how RISC-V works, its architecture, and why it's becoming a key player in modern processor design.

